

基于 LSTM 的中文文本分类与情感分析对比研究

摘要

本研究以 THUCNews 中文新闻数据集为对象，结合文本分类与情感分析进行对比研究。文本分类方面，采用 Word2Vec 词向量结合 LSTM 神经网络作为主模型，以 Naive Bayes + TF-IDF 作为基线对比；情感分析方面，基于大连理工情感词汇本体库（DUTIR）进行词典级情感极性判定。实验结果表明，LSTM 模型在十类别新闻分类任务上取得了 88.68% 的测试准确率，高于 Naive Bayes 基线（29.67%）。进一步诊断发现，该基线的词元切分配置不适合中文：其特征提取在未分词的原始标题上使用面向英文的切分规则，每条文本仅切出 1.97 个词元；改用 jieba 分词后，基线在同等训练量下降至 84.19%，在全集 180,000 条上达到 86.58%，两者差距相应收窄至 4.49 与 2.10 个百分点，说明原始对比显著高估了深度学习的增量。情感分析揭示了不同新闻类别的情感分布差异：体育和娱乐类正面情感突出（69.4%、64.2%），社会和政治类负面情感占比较高（42.2%、34.8%）；但类别分类准确率与正面情感占比之间未发现稳定关联（Pearson $r = 0.182$, $p = 0.615$ ）。

关键词：文本分类；情感分析；LSTM；Word2Vec；THUCNews

1. 引言

随着互联网新闻数据的爆炸式增长，如何有效地对海量文本进行自动分类与情感分析已成为自然语言处理领域的重要课题。文本分类旨在将文档自动划分到预定义的类别中，而情感分析则关注文本所表达的情感倾向。二者在舆情监测、信息推荐、智能问答等场景中均有广泛应用。

近年来，基于深度学习的文本分类方法取得了显著进展。其中，LSTM 因其对序列数据的建模能力，在文本分类任务中表现出色。与此同时，基于情感词典的方法因其无需标注数据和计算资源较少的特点，仍被广泛用于情感分析任务。

本研究以 THUCNews 中文新闻数据集为对象，以 LSTM 文本分类为主任务，Naive Bayes + TF-IDF 为基线对比，同时使用大连理工情感词汇本体库（DUTIR）进行情感分析，最终将分类结果与情感分析结果进行对比讨论。本文旨在探究深度学习方法与传统方法在中文文本分类上的差异，并分析不同新闻类别的情感分布特征。

2. 数据与方法

2.1. THUCNews 数据集

本文采用清华大学自然语言处理与社会人文计算实验室发布的 THUCNews 中文新闻数据集的十类别子集，包含财经、房产、股票、教育、科技、社会、政治、体育、游戏、娱乐共 10 个新闻类别。原始数据来源于新浪新闻 RSS 订阅频道，经人工标注后用于文本分类研究。本文所使用的十类别子集共 200,000 条样本（每类 20,000 条），按照 9:0.5:0.5 的比例划分为训练集 180,000 条、验证集 10,000 条和测试集 10,000 条，各类别样本数量均衡。该数据集在本文实验中通过 Kaggle 平台获取（<https://www.kaggle.com/datasets/xianhuizhang/thucnews>）。

2.2. 数据预处理

预处理流程如下：首先对文本进行清洗，保留中文字符、英文字母和数字；接着使用 jieba 分词工具进行中文分词，并去除 746 个停用词；然后基于训练集构建词汇表，设定最大词汇量为 50,000，最低词频为 5，最终词汇表大小为 30,351；最后将文本序列编码为长度 200 的索引序列，以 <PAD> 填充不足部分（LSTM 训练时进一步截断至 50，因实际最大长度仅 15 个词，不损失信息）。

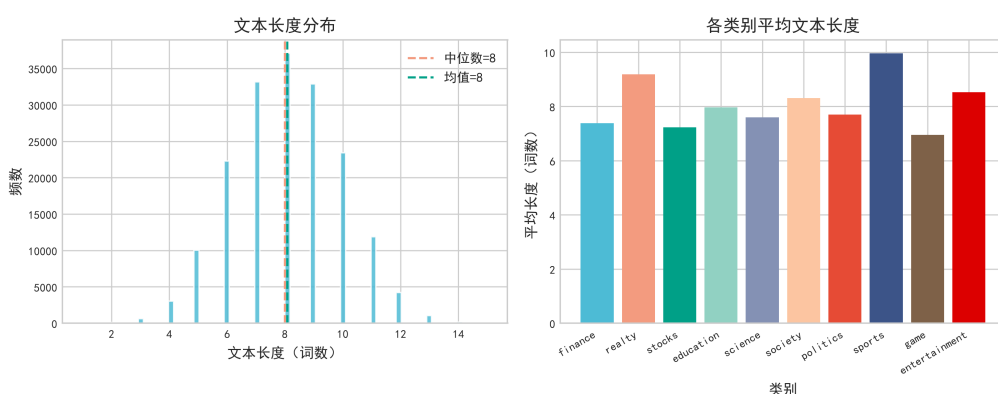


图 1: 文本长度分布（左）与各类别平均文本长度（右）

图 1 展示了训练集的文本长度分布情况。文本长度的均值为 8.08 个词，中位数为 8 个词，标准差 1.85，最大长度为 15 个词，表明 THUCNews 数据集的文本以短文本为主，符合新闻标题的特征。各类别的平均文本长度存在一定差异：体育类最长（9.96 个词），游戏类最短（6.96 个词），相差约 43%。

2.3. Word2Vec 词向量

本文采用 Word2Vec Skip-gram 模型在训练集上进行无监督词向量训练。词向量维度设为 200，上下文窗口为 5，最低词频为 5，采用负采样数为 10 的策略，迭代 10 轮。训练共耗时 56.6 秒，获得 30,349 个词的向量表示，覆盖词汇表全部词语。预训练的词向量作为 LSTM 嵌入层的初始权重。

2.4. LSTM 文本分类模型

LSTM 是一种特殊的循环神经网络，通过引入遗忘门、输入门和输出门的门控机制，有效缓解了传统循环神经网络中的梯度消失和梯度爆炸问题。对于文本分类任务，LSTM 能够捕捉文本中的长距离依赖关系，提取序列的上下文特征。

本文搭建的 LSTM 分类模型结构如下：首先使用预训练的 Word2Vec 词向量初始化嵌入层，嵌入层在训练过程中进行微调；接着接单层 LSTM 层，隐藏单元数分别为 64 和 128；然后对 LSTM 所有时间步的输出进行基于掩码的平均池化，以聚合序列信息；最后通过丢弃率为 0.5 的 Dropout 层和全连接层输出 10 个类别的概率分布。

模型训练采用 Adam 优化器，学习率为 0.01，批大小为 128，最大训练轮次为 20，并采用早停策略，若验证损失连续 4 轮不下降则停止训练。为加速训练，对训练集按类别等比例采样至每类 3,000 条，共计 30,000 条。

2.5. 情感分析方法

本文采用基于情感词典的方法进行情感分析，具体使用大连理工大学信息检索研究室构建的情感词汇本体库（DUTIR），通过 Python 的 cnsenti 库调用。该词典包含 27,000 余个情感词汇，涵盖乐、好、怒、哀、惧、恶、惊 7 大类情感，并标注了情感强度与极性。

情感分析流程为：对每条文本统计其中出现的正面情感词和负面情感词数量，根据正负词数的比较判定文本的情感极性为正面、负面或中性。

3. 实验与结果

3.1. 文本分类结果

表 1: 分类模型性能对比

模型	准确率	F1 (macro)	训练时间
LSTM (h=64, d=0.5)	0.8868	0.8867	85.7s
LSTM (h=128, d=0.5)	0.8768	0.8770	135.3s
Naive Bayes + TF-IDF	0.2967	0.2944	3.1s

表 1 展示了各模型在测试集上的分类性能。LSTM 模型在两个配置下均取得了接近 88% 的分类准确率，其中隐藏单元数为 64 的配置表现最佳，测试准确率为 0.8868，F1 值为 0.8867。

需要指出的是，Naive Bayes 基线的实现存在一处配置缺陷：其特征提取调用未分词的原始标题，却使用面向英文的词元切分规则，而中文标题不含空格，因此每条文本仅被切出 1.97 个词元（jieba 分词后为 8.05 个），模型实际是在整条标题而非词上建模，导致 10,000 条测试样本中有 7,655 条被预测为同一类别。改用 jieba 分词后，基线在同

等 30,000 条训练量下准确率升至 84.19%，在全集 180,000 条上达到 86.58%。因此两个模型之间的真实差距约为 4.49 个百分点（同等训练量）或 2.10 个百分点（对全集基线），而非表中呈现的约 59 个百分点。

LSTM 模型在两个配置上均在第 5 轮触发早停，且最佳验证准确率都出现在第 1 轮（分别为 0.8733 与 0.8644），表明预训练词向量已提供较强的初始特征表示，此后训练主要表现在训练集上的过拟合。隐藏单元数从 64 增加到 128 并未带来性能提升，反而使训练时间增加约 58%。

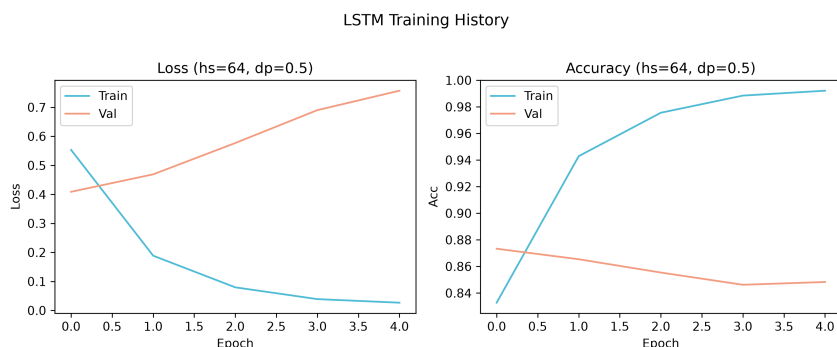
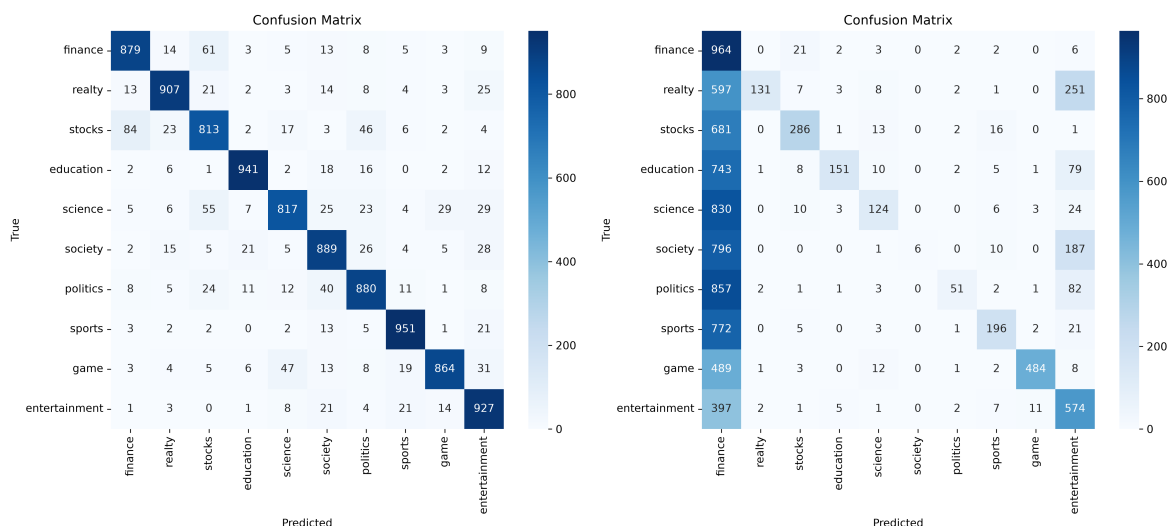


图 2: LSTM 最佳模型训练曲线

图 2 显示了 LSTM 最佳模型的训练过程。模型在第一个 epoch 后训练准确率即达 83.27%，验证准确率达 87.33%，表明预训练的 Word2Vec 词向量提供了良好的初始特征表示。随后训练准确率持续上升至 99% 以上，但验证准确率开始缓慢下降，出现明显过拟合，早停在第 5 轮触发。



(a) LSTM

(b) Naive Bayes

图 3: LSTM 与 Naive Bayes 混淆矩阵对比

图 3 展示了 LSTM 与 Naive Bayes 模型在测试集上的混淆矩阵。LSTM 的混淆矩阵呈现清晰的 diagonal 结构，分类错误主要集中在语义相近的类别之间，如财经与股

票、社会与政治等。相比之下，NB 的预测几乎完全塌缩到单一类别：10,000 条测试样本中有 7,655 条被判为财经类，绝大多数类别的召回率接近零。如 3.1 节所述，这一失效模式主要来自词元切分配置，而非朴素贝叶斯的特征独立性假设本身。

3.2. 情感分析结果

对测试集 10,000 条样本进行情感分析的结果显示，正面情感样本占比 43.3%，中性样本占 41.7%，负面样本占 15.0%。这表明 THUCNews 数据集整体上以正面和中性新闻为主，负面新闻占比较低。

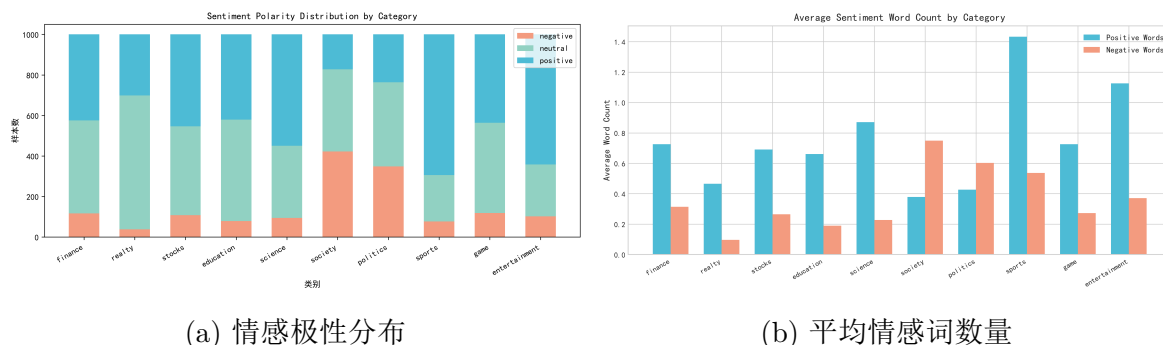


图 4: 情感分析结果

图 4a 显示了各类别的情感极性分布。体育的正面比例最高，达 69.4%；社会的正面比例最低，仅 17.2%，但其负面比例最高，达 42.2%。房产的中性比例高达 66.1%，是情感特征最不鲜明的类别。图 4b 从情感词数量的角度呈现了相同趋势。不同新闻类别的情感特征呈现出明显差异：体育和娱乐类别的正面情感词数量最高，这与体育赛事和娱乐新闻的正面报道特征相符；社会和政治类别的负面情感词数量最高，反映了这两类新闻中包含较多的负面事件报道。值得注意的是，房产类别的情感词总量最低，这可能与房产新闻多采用客观数据报道的风格有关。

进一步观察情感词数量的分布特征（图 5）可以发现，正面情感词在各类别中的分布较为集中，而负面情感词则呈现出更强的类别特异性——社会和政治类别的负面情感词离群值较多、分布范围更广，说明这两类新闻不仅在平均负面情感词数量上偏高，且负面情感的出现在文本之间具有较大的波动性。

3.3. 分类与情感分析对比

一个自然的猜想是：分类准确率较高的类别是否同时也是正面情感占比最高的类别？本文用保存的最佳模型复算了 10 个类别的分类准确率，并与各类别的正面情感占比做相关分析，结果为 Pearson $r = 0.182$ ($p = 0.615$)、Spearman $\rho = 0.042$ ($p = 0.907$)，均不显著。数据中存在明确反例：科技类的正面情感占比为 55.0%（第三高），分类准确率却只有 0.817；社会类的正面情感占比最低（17.2%），分类准确率反而达到 0.889。因此本文认为，类别的可分类性主要由类间词汇重叠程度决定，与情感倾向之间未发现稳定关联。

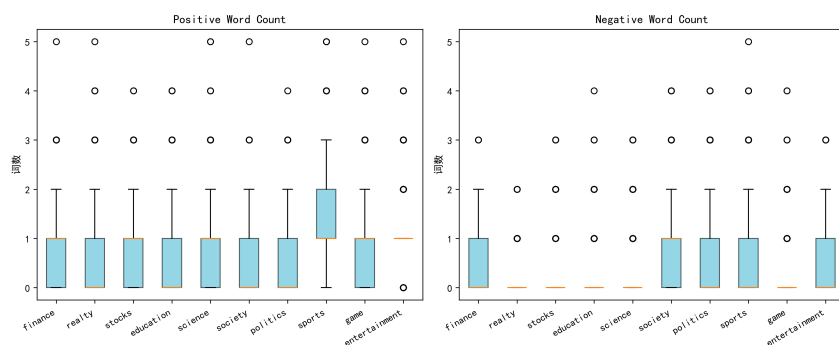


图 5: 各类别情感词数量分布

尽管如此，LSTM 模型的高分类准确率（88.68%）与基于词典的情感分析仍构成一组有价值的对照。前者通过有监督学习自动提取文本特征，后者则依赖人工构建的情感词典进行无监督分析，两者结合可以更全面地刻画文本数据的特征。

4. 结论

本文以 THUCNews 中文新闻数据集为对象，开展了一项结合文本分类与情感分析的对比研究。在文本分类方面，LSTM 模型取得了 88.68% 的测试准确率，明显高于 Naive Bayes + TF-IDF 基线的 29.67%；但在纠正基线的词元切分配置后，前者相对后者的优势收窄至 4.49 个百分点（同等训练量）与 2.10 个百分点（对全集基线），说明深度学习在中文短文本分类上的增量远小于原始对比所显示的程度，而基线实现的正确性对结论具有决定性影响。在情感分析方面，基于 DUTIR 情感词典的分析揭示了不同新闻类别的情感分布差异，体育和娱乐类正面情感突出，社会和政治类负面情感较多；不过分类准确率与正面情感占比之间未发现稳定关联。

本研究的不足在于：(1) LSTM 模型仅在 30,000 条采样数据上训练，若能使用全量数据并结合 GPU 加速，预期可进一步提升性能；(2) 情感分析仅采用词典方法，缺少情感标注因而无法评估其准确率，也未与基于模型的方法进行对比；(3) 最佳超参数配置由测试集准确率挑选，存在轻微的信息泄漏；(4) 基线缺陷的发现说明实验结论对实现细节高度敏感，后续工作应把流水线自检作为标准步骤。后续工作可考虑使用预训练语言模型（如 BERT）进行文本分类，并探索基于深度学习的情感分析模型。

A. 工具声明

本文的实验代码、调试与排版通过 AI 编程助手辅助完成:正文所依赖的 `scripts/01-06` 与本文初稿在 Claude Code (后端 DeepSeek v4 Pro) 辅助下完成; 用于核验正文全部数值、诊断基线词元切分配置并重跑全流程的 `scripts/07_baseline_diagnosis.py` 由 DeepSeek Harness 完成。研究问题、方法选型、实验方案与结果解读由作者负责。

B. 数据说明

- 数据集: THUCNews (清华中文新闻分类数据集, 10 个类别)
- 分类方法: LSTM + Word2Vec 词嵌入, 需调参
- 情感分析: 基于 DUTIR 中文情感词典
- 数据来源: Kaggle (<https://www.kaggle.com/datasets/xianhuizhang/thucnews>)

C. 提示词

以下为项目进行过程中使用的关键提示词以及决策:

我想要用 LSTM 做中文文本分类和情感分析, 并且要可视化结果, 请你为我推荐合适的数据集和一些相关的扩展性分析供我选择。请帮我写代码实现这个任务, 并且在代码中加入一些注释说明。

弹窗决策及理由

决策	理由
项目目录为当前文件夹	用户指定
方向: 文本分类 + 情感分析 + 可视化对比	用户指定
数据集: THUCNews (10 类)	中文新闻分类, 支持多类别任务
分类模型: LSTM + Word2Vec	深度学习序列模型, 适合文本分类
基线对比: Naive Bayes + TF-IDF	仅作对比, 不占过多篇幅
情感分析: cnsenti (DUTIR 词典)	学术权威, pip 安装即用
分词工具: jieba	中文分词最常用工具
词向量: 自训练 Word2Vec	灵活性高, 便于调参

D. 核心代码

D.1. LSTM 分类模型

```
import os, sys, pickle, time, copy, random
import numpy as np
import torch
```

```

import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, TensorDataset
from sklearn.feature_extraction.text import CountVectorizer, TfidfTransformer
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score, f1_score
import matplotlib
import matplotlib.font_manager as fm
for _fp in [r'C:\Windows\Fonts\simhei.ttf', r'C:\Windows\Fonts\msyh.ttc']:
    if os.path.exists(_fp):
        fm.fontManager.addfont(_fp)
matplotlib.rcParams['axes.unicode_minus'] = False
import matplotlib.pyplot as plt

ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
PROCESSED_PATH = os.path.join(ROOT, 'data', 'processed', 'processed.pkl')
EMB_PATH = os.path.join(ROOT, 'output', 'models', 'embedding_matrix.npy')
FIGURE_DIR = os.path.join(ROOT, 'output', 'figures')
TABLE_DIR = os.path.join(ROOT, 'output', 'tables')
MODEL_DIR = os.path.join(ROOT, 'output', 'models')
os.makedirs(FIGURE_DIR, exist_ok=True)
os.makedirs(TABLE_DIR, exist_ok=True)
os.makedirs(MODEL_DIR, exist_ok=True)
random.seed(42); np.random.seed(42); torch.manual_seed(42)

HIDDEN_SIZES = [64, 128]
DROPOUTS = [0.5]
BATCH_SIZE = 128
LR = 0.01
NUM_EPOCHS = 20
PATIENCE = 4
SUBSAMPLE = 3000
SEQ_LENGTH = 50
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

class LSTMClassifier(nn.Module):
    def __init__(self, vocab_size, emb_dim, hidden_size,
                  num_layers, num_classes, dropout, pretrained_emb):
        super().__init__()
        self.embedding = nn.Embedding.from_pretrained(
            pretrained_emb, freeze=False, padding_idx=0)
        self.lstm = nn.LSTM(emb_dim, hidden_size, num_layers, batch_first=True)
        self.dropout = nn.Dropout(dropout)
        self.fc = nn.Linear(hidden_size, num_classes)

```

```

def forward(self, x):
    emb = self.embedding(x)
    lstm_out, _ = self.lstm(emb)
    mask = (x != 0).unsqueeze(-1).float()
    pooled = (lstm_out * mask).sum(dim=1) / mask.sum(dim=1).clamp(min=1)
    return self.fc(self.dropout(pooled))

def train_epoch(model, loader, optimizer, criterion):
    model.train()
    total_loss, correct, total = 0, 0, 0
    for x, y in loader:
        x, y = x.to(device), y.to(device)
        optimizer.zero_grad()
        logits = model(x)
        loss = criterion(logits, y)
        loss.backward()
        optimizer.step()
        total_loss += loss.item() * x.size(0)
        correct += (logits.argmax(1) == y).sum().item()
        total += x.size(0)
    return total_loss / total, correct / total

@torch.no_grad()
def evaluate(model, loader, criterion):
    model.eval()
    total_loss, correct, total = 0, 0, 0
    all_preds, all_labels = [], []
    for x, y in loader:
        x, y = x.to(device), y.to(device)
        logits = model(x)
        loss = criterion(logits, y)
        total_loss += loss.item() * x.size(0)
        preds = logits.argmax(1)
        correct += (preds == y).sum().item()
        total += x.size(0)
        all_preds.extend(preds.cpu().numpy())
        all_labels.extend(y.cpu().numpy())
    return total_loss / total, correct / total, all_preds, all_labels

if __name__ == '__main__':
    with open(PROCESSED_PATH, 'rb') as f:
        data = pickle.load(f)
    train_enc, train_labels = data['train_encoded'], data['train_labels']
    dev_enc, dev_labels = data['dev_encoded'], data['dev_labels']
    test_enc, test_labels = data['test_encoded'], data['test_labels']

```

```

train_texts = data['train_texts']
class_names = data['class_names']
emb_matrix = np.load(EMB_PATH)

train_sub_enc, train_sub_labels = [], []
for lb in range(len(class_names)):
    chosen = random.sample([i for i, l in enumerate(train_labels) if l == lb],
                           min(SUBSAMPLE, (train_labels == lb).sum()))
    train_sub_enc.extend([train_enc[i] for i in chosen])
    train_sub_labels.extend([train_labels[i] for i in chosen])

def make_loader(x, y, shuffle=False):
    x = [seq[:SEQ_LENGTH] for seq in x]
    return DataLoader(TensorDataset(torch.LongTensor(x), torch.LongTensor(y)),
                      batch_size=BATCH_SIZE, shuffle=shuffle)

train_loader = make_loader(train_sub_enc, train_sub_labels, shuffle=True)
dev_loader = make_loader(dev_enc, dev_labels)
test_loader = make_loader(test_enc, test_labels)

for hidden_size in HIDDEN_SIZES:
    for dropout in DROPOUTS:
        model = LSTMClassifier(emb_matrix.shape[0], emb_matrix.shape[1],
                               hidden_size, 1, len(class_names), dropout,
                               torch.FloatTensor(emb_matrix)).to(device)
        criterion = nn.CrossEntropyLoss()
        optimizer = optim.Adam(model.parameters(), lr=LR)
        best_val_loss = float('inf'); patience = 0
        for epoch in range(1, NUM_EPOCHS + 1):
            train_loss, train_acc = train_epoch(model, train_loader, optimizer,
                                                  criterion)
            val_loss, val_acc, _, _ = evaluate(model, dev_loader, criterion)
            if val_loss < best_val_loss:
                best_val_loss = val_loss; patience = 0
                best_state = copy.deepcopy(model.state_dict())
            else:
                patience += 1
                if patience >= PATIENCE: break
        model.load_state_dict(best_state)
        _, test_acc, preds, true = evaluate(model, test_loader, criterion)
        print(f'h={hidden_size}_test_acc={test_acc:.4f}_
              f'f1={f1_score(true, preds, average="macro"):.4f}')

vec = CountVectorizer(max_features=10000)
X_train = TfidfTransformer().fit_transform(vec.fit_transform(train_texts))

```

```

X_test = TfidfTransformer().fit_transform(vec.transform(test_texts))
nb = MultinomialNB().fit(X_train, train_labels)
nb_preds = nb.predict(X_test)
print(f'NB: acc={accuracy_score(test_labels, nb_preds):.4f} '
      f'f1={f1_score(test_labels, nb_preds, average="macro"): .4f}')

```

D.2. 情感分析

```

import os, pickle, time
import pandas as pd
from cnsenti import Sentiment
import matplotlib
import matplotlib.font_manager as fm
for _fp in [r'C:\Windows\Fonts\simhei.ttf', r'C:\Windows\Fonts\msyh.ttc']:
    if os.path.exists(_fp):
        fm.fontManager.addfont(_fp)
matplotlib.rcParams['font.sans-serif'] = ['SimHei', 'Microsoft YaHei']
matplotlib.rcParams['axes.unicode_minus'] = False
import matplotlib.pyplot as plt

ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
PROCESSED_PATH = os.path.join(ROOT, 'data', 'processed', 'processed.pkl')
FIGURE_DIR = os.path.join(ROOT, 'output', 'figures')
os.makedirs(FIGURE_DIR, exist_ok=True)

def analyze_sentiment_batch(texts, batch_size=500):
    sent = Sentiment()
    results = []
    n = len(texts)
    for i in range(0, n, batch_size):
        for text in texts[i:i + batch_size]:
            results.append(sent.sentiment_count(text))
    return results

def classify_sentiment(pos, neg):
    return 'positive' if pos > neg else ('negative' if neg > pos else 'neutral')

if __name__ == '__main__':
    with open(PROCESSED_PATH, 'rb') as f:
        data = pickle.load(f)
    test_texts = data['test_texts']
    test_labels = data['test_labels']
    class_names = data['class_names']

    results = analyze_sentiment_batch(test_texts)

```

```

rows = []
for i, res in enumerate(results):
    rows.append({'label': test_labels[i], 'class_name': class_names[test_labels[i]
    ]},
                'pos': res['pos'], 'neg': res['neg'],
                'sentiment': classify_sentiment(res['pos'], res['neg'])})
df = pd.DataFrame(rows)
df.to_pickle(os.path.join(ROOT, 'data', 'processed', 'sentiment_results.pkl'))

total = len(df)
pos = (df['sentiment'] == 'positive').sum()
neg = (df['sentiment'] == 'negative').sum()
neu = (df['sentiment'] == 'neutral').sum()
print(f'pos={pos}({pos/total*100:.1f}%)_neu={neu}({neu/total*100:.1f}%)_'
      f'neg={neg}({neg/total*100:.1f}%)')

for cn in class_names:
    sub = df[df['class_name'] == cn]
    p = (sub['sentiment'] == 'positive').sum()
    n = (sub['sentiment'] == 'negative').sum()
    print(f'{cn}:_pos={p}_neg={n}')

```